

INTERN ONBOARDING

2-Week AI Engineering Bootcamp

Python · ReactJS · RAG · MCP · LangChain · LangGraph · Prompting

Learning Schedule, Resources & Self-Study Prompts

Working hours: Mon–Fri · 10:30 AM – 6:30 PM (Lunch 1:30–2:30 PM)

Goal: After two weeks, interns begin building AI agents and systems using AI tools like Claude Code.

Contents

Contents.....	2
1. Program Overview	3
Daily Structure.....	3
Learning Outcomes.....	3
Weekly Theme	3
2. Week 1 — Foundations & Front-end	5
Day 1 (Monday) — Python Refresher & Environment Setup	5
Day 2 (Tuesday) — Python for AI: APIs, Async & Libraries.....	6
Day 3 (Wednesday) — ReactJS Fundamentals.....	7
Day 4 (Thursday) — React State, Hooks & API Integration.....	8
Day 5 (Friday) — Prompting Fundamentals + Week 1 Assessment.....	9
3. Week 2 — AI Systems & Orchestration	10
Day 6 (Monday) — RAG: Retrieval-Augmented Generation.....	10
Day 7 (Tuesday) — LangChain.....	11
Day 8 (Wednesday) — LangGraph & Agents	12
Day 9 (Thursday) — MCP & MCP Servers	13
Day 10 (Friday) — Capstone Integration + Week 2 Assessment.....	14
4. Friday Assessment Program.....	15
Week 1 Assessment — Friday, Day 5 (10:30 AM – 1:30 PM)	15
Week 2 Assessment — Friday, Day 10 (10:30 AM – 1:30 PM)	16
5. Learning with Claude Code.....	17
Setup	17
How to Learn with Claude Code.....	17
Reusable Tutor Prompt Template	17
Debugging Prompt Template	18
6. After the Bootcamp — The Agent-Building Phase.....	19
What Changes.....	19
Readiness Checklist.....	19

1. Program Overview

This is a two-week, full-time bootcamp designed to take coding-literate interns and equip them with the modern AI engineering stack. The interns already know how to code, so the focus is on AI-specific tooling, frameworks, and patterns rather than programming fundamentals.

Daily Structure

Every working day follows the same rhythm so interns can build a predictable study habit:

Time	Block	Purpose
10:30 – 11:30 AM	Concept Session	Reading, videos, theory — understand the 'why'
11:30 AM – 1:30 PM	Hands-on Lab	Build along — code the concept yourself
1:30 – 2:30 PM	Lunch Break	Rest and recharge
2:30 – 4:30 PM	Guided Project	Apply the concept to a small project
4:30 – 6:00 PM	Independent Practice	Extend the project / solve exercises
6:00 – 6:30 PM	Daily Wrap-up	Notes, commit code, log blockers & questions

How to use this document

Each day lists a Concept, a Lab, and a Project. Follow them in order.

Every topic has Learning Sources (free, high-quality) and a Claude Code Prompt.

If a concept is unclear, paste the provided prompt into Claude Code and learn interactively.

Commit your code to a personal repo daily — Friday assessments are practical.

Learning Outcomes

By the end of Week 2, each intern should be able to:

- Write clean, idiomatic Python and manage environments & dependencies confidently.
- Build interactive front-ends in ReactJS with components, state, and hooks.
- Design effective prompts and understand model behaviour, context, and limitations.
- Build a Retrieval-Augmented Generation (RAG) pipeline from documents to answers.
- Compose LLM applications using LangChain and orchestrate multi-step agents with LangGraph.
- Understand the Model Context Protocol (MCP) and build a working MCP server.
- Use Claude Code as a daily engineering partner to plan, build, and debug.

Weekly Theme

Week	Theme	Topics Covered
Week 1	Foundations & Front-end	Python refresher, ReactJS, Prompting fundamentals
Week 2	AI Systems & Orchestration	RAG, LangChain, LangGraph, MCP & MCP Servers

2. Week 1 — Foundations & Front-end

Week 1 establishes the two pillars every AI project rests on: a solid Python base for back-end and AI logic, a working knowledge of ReactJS for building interfaces, and the prompting skill that connects humans to models.

Day 1 (Monday) — Python Refresher & Environment Setup

DAY 1 · MONDAY — Python Core	
10:30–11:30	Concept: Python data types, comprehensions, functions, error handling. Review what makes Python idiomatic (PEP 8, the Zen of Python).
11:30–1:30	Lab: Set up Python 3.11+, VS Code, and virtual environments. Install Claude Code and verify it runs. Practice list/dict comprehensions and f-strings.
2:30–4:30	Project: Build a command-line 'expense tracker' — add, list, filter, and summarise expenses stored in a JSON file.
4:30–6:00	Practice: Add input validation and exception handling. Refactor into clean functions.
6:00–6:30	Wrap-up: Push code to GitHub. Note any Python concepts that felt shaky.

Learning Sources

- [Official Python Tutorial](#) — sections 3–9 (the canonical reference).
- [Real Python — Python Basics](#) — practical, well-explained articles.
- [Python Virtual Environments Primer](#) — venv, pip, and dependency hygiene.
- [freeCodeCamp — Python Full Course \(YouTube\)](#) — for anyone wanting a video walkthrough.

Claude Code Prompt — Day 1

I am an intern who already knows programming but want to refresh modern Python. Act as my Python mentor. Quiz me on data types, comprehensions, functions, and exception handling — one question at a time. After each answer, give feedback and a short idiomatic-Python tip. Then walk me through building a command-line expense tracker that stores data in a JSON file, explaining each design decision. Don't write all the code at once — guide me step by step and let me write each function first.

Day 2 (Tuesday) — Python for AI: APIs, Async & Libraries

DAY 2 · TUESDAY — Python for AI	
10:30–11:30	Concept: Working with REST APIs, JSON, environment variables (.env), and the basics of async/await. Intro to pip packages used in AI work (requests, pydantic, python-dotenv).
11:30–1:30	Lab: Call a public API with 'requests'. Parse JSON responses. Store an API key safely in a .env file.
2:30–4:30	Project: Build a script that fetches data from a public API (e.g. weather or news), validates it with Pydantic, and saves a clean report.
4:30–6:00	Practice: Add retry logic and rate-limit handling. Convert one function to async.
6:00–6:30	Wrap-up: Commit code. List which libraries you used and what each does.

Learning Sources

- [Real Python — requests Library Guide](#) — HTTP in Python.
- [Pydantic Documentation](#) — data validation, used heavily across AI frameworks.
- [Real Python — Async IO in Python](#) — async/await explained clearly.
- [Twelve-Factor App — Config](#) — why secrets belong in environment variables.

Claude Code Prompt — Day 2

Teach me how Python talks to the outside world for AI projects. Cover: making HTTP requests with the requests library, parsing JSON, loading secrets from a .env file, and validating data with Pydantic. Explain async/await with a simple analogy first. Then guide me to build a script that fetches data from a free public API, validates the response with a Pydantic model, and writes a clean summary report. Point out security mistakes if I hard-code any API keys.

Day 3 (Wednesday) — ReactJS Fundamentals

DAY 3 · WEDNESDAY — React Basics	
10:30–11:30	Concept: What React is and why it exists. JSX, components, props, and the component tree. Setting up a project with Vite.
11:30–1:30	Lab: Create a Vite + React app. Build static components and pass data via props. Style with plain CSS or Tailwind.
2:30–4:30	Project: Build a 'profile card gallery' — a reusable Card component rendered from an array of data.
4:30–6:00	Practice: Make the layout responsive. Split the UI into small, well-named components.
6:00–6:30	Wrap-up: Commit code. Note the difference between props and state (preview for Day 4).

Learning Sources

- [React Official Docs — Learn React](#) — the modern, definitive tutorial.
- [React Docs — Quick Start](#) — hands-on tic-tac-toe project.
- [Vite — Getting Started](#) — fast modern React tooling.
- [Scrimba — Learn React \(free course\)](#) — interactive video lessons.

Claude Code Prompt — Day 3

I know programming but I'm new to React. Explain React from first principles: what problem it solves, what JSX is, and how components and props work. Use a concrete analogy for the component tree. Then guide me to scaffold a Vite + React project and build a 'profile card gallery' where a reusable Card component is rendered from an array of objects. Review my JSX for bad habits and explain naming conventions for components.

Day 4 (Thursday) — React State, Hooks & API Integration

DAY 4 · THURSDAY — React Interactive	
10:30–11:30	Concept: State and the useState hook. Event handling. Side effects with useEffect. Fetching data from an API inside a component.
11:30–1:30	Lab: Build interactive components — a counter, a controlled form, and a toggle. Use useEffect to fetch data on mount.
2:30–4:30	Project: Build a small app that fetches from a public API and displays results, with loading and error states.
4:30–6:00	Practice: Add a search/filter box. Lift state up where needed. Handle empty results gracefully.
6:00–6:30	Wrap-up: Commit code. Prepare — Friday is the Week 1 assessment (first 3 hours).

Learning Sources

- [React Docs — State: A Component's Memory](#) — useState explained.
- [React Docs — Synchronizing with Effects](#) — useEffect done right.
- [React Docs — Reacting to Input with State](#) — controlled forms.
- [Fetching Data in React \(patterns\)](#) — loading & error handling.

Claude Code Prompt — Day 4

Teach me React state and hooks. Start with useState (what 'state' means and how it differs from props), then event handling, then useEffect for side effects. After explaining each, give me a tiny exercise to write myself before moving on. Finally, guide me to build an app that fetches data from a public API on mount, shows loading and error states, and has a search box that filters results. Review my code for the most common React mistakes (stale state, missing keys, wrong dependency arrays).

Day 5 (Friday) — Prompting Fundamentals + Week 1 Assessment

Friday Format (applies to both Fridays)

10:30 AM – 1:30 PM → Assessment (first 3 hours, practical, no written question paper).

2:30 PM – 6:30 PM → New learning topic for the day, as scheduled below.

The assessment is a hands-on build. Interns are evaluated on what they ship, not memorised answers.

DAY 5 · FRIDAY — Assessment + Prompting

10:30–1:30	WEEK 1 ASSESSMENT: Practical build combining Python + React (see Section 4).
2:30–3:45	Concept: Prompting fundamentals — how LLMs work, tokens, context windows, system vs user prompts, temperature.
3:45–5:30	Lab: Practice prompt patterns — role prompting, few-shot examples, chain-of-thought, structured output (JSON).
5:30–6:30	Wrap-up: Build a small prompt library. Document which patterns worked best and why.

Learning Sources

- [Anthropic — Prompt Engineering Overview](#) — the core reference for this program.
- [Anthropic — Prompt Engineering Interactive Tutorial](#) — hands-on, highly recommended.
- [OpenAI — Prompt Engineering Guide](#) — complementary techniques.
- [Learn Prompting](#) — free, structured prompting course.

Claude Code Prompt — Day 5

Be my prompting instructor. First explain how LLMs process text: tokens, context windows, and why phrasing matters. Then teach these patterns one at a time with examples I can try: role prompting, few-shot examples, chain-of-thought, and asking for structured JSON output. After each, give me a task and critique the prompt I write. End by helping me assemble a reusable prompt library file with my best prompts and notes on when to use each.

3. Week 2 — AI Systems & Orchestration

Week 2 moves from foundations to building real AI systems. Interns learn how to give models access to private knowledge (RAG), how to compose multi-step LLM applications (LangChain), how to orchestrate agents (LangGraph), and how the Model Context Protocol standardises tool access.

Day 6 (Monday) — RAG: Retrieval-Augmented Generation

DAY 6 · MONDAY — RAG Foundations	
10:30–11:30	Concept: Why RAG exists — LLM knowledge limits & hallucination. Embeddings, vector similarity, chunking, and vector databases.
11:30–1:30	Lab: Generate embeddings for text. Store them in a vector store (Chroma / FAISS). Run similarity search queries.
2:30–4:30	Project: Build a 'chat with your documents' pipeline — load a PDF, chunk it, embed it, retrieve relevant chunks, and answer questions.
4:30–6:00	Practice: Experiment with chunk size and overlap. Compare answer quality. Add source citations.
6:00–6:30	Wrap-up: Commit code. Write 3 sentences on what made retrieval better or worse.

Learning Sources

- [What is RAG? — AWS Explainer](#) — clear conceptual intro.
- [LangChain — RAG Tutorial](#) — official end-to-end RAG build.
- [Chroma — Getting Started](#) — a simple, popular vector database.
- [Pinecone — Embeddings & Vector Search Guide](#) — embeddings explained well.

Claude Code Prompt — Day 6

Teach me Retrieval-Augmented Generation from scratch. First explain the problem RAG solves and the key ideas: embeddings, vector similarity, chunking, and vector databases – use an analogy for embeddings. Then guide me to build a 'chat with your documents' pipeline: load a PDF, split it into chunks, embed them, store them in Chroma, retrieve the most relevant chunks for a question, and pass them to an LLM to answer with citations. Explain each parameter (chunk size, overlap, top-k) and have me experiment with them.

Day 7 (Tuesday) — LangChain

DAY 7 · TUESDAY — LangChain	
10:30–11:30	Concept: What LangChain is — models, prompts, output parsers, and chains. The LangChain Expression Language (LCEL) and how components compose.
11:30–1:30	Lab: Build basic chains — a prompt + model + parser. Use output parsers to get structured data.
2:30–4:30	Project: Rebuild Day 6's RAG pipeline using LangChain components (loaders, splitters, retrievers, chains) for a cleaner architecture.
4:30–6:00	Practice: Add memory / conversation history. Swap components and observe the effect.
6:00–6:30	Wrap-up: Commit code. Note how LangChain changed the structure vs the manual version.

Learning Sources

- [LangChain — Introduction](#) — official overview.
- [LangChain — Build a Simple LLM Application](#) — first chain tutorial.
- [LangChain Expression Language \(LCEL\)](#) — how components compose.
- [LangChain — Chatbot Tutorial](#) — adding memory.

Claude Code Prompt — Day 7

Introduce me to LangChain. Explain its core building blocks – models, prompt templates, output parsers, and chains – and what the LangChain Expression Language (LCEL) is and why the pipe (|) syntax exists. Guide me to build a simple chain first (prompt + model + structured parser). Then help me refactor a manual RAG pipeline I built earlier into clean LangChain components: document loader, text splitter, vector store retriever, and a retrieval chain. Finally add conversation memory and explain the trade-offs.

Day 8 (Wednesday) — LangGraph & Agents

DAY 8 · WEDNESDAY — LangGraph	
10:30–11:30	Concept: Why agents need graphs. State, nodes, edges, and conditional routing. The difference between a chain (linear) and a graph (branching, looping).
11:30–1:30	Lab: Build a minimal LangGraph — define state, add nodes, connect edges, and run it. Add a conditional edge.
2:30–4:30	Project: Build a tool-using agent — the agent decides whether to call a tool (e.g. search or calculator) and loops until it can answer.
4:30–6:00	Practice: Add a second tool. Add a node that reflects on the answer before finishing.
6:00–6:30	Wrap-up: Commit code. Sketch your agent's graph (nodes & edges) in your notes.

Learning Sources

- [LangGraph — Documentation](#) — official docs & concepts.
- [LangGraph — Quickstart](#) — build your first graph.
- [LangGraph — Agentic Concepts](#) — how agents are modelled.
- [LangChain Academy — Intro to LangGraph \(free\)](#) — structured free course.

Claude Code Prompt — Day 8

Teach me LangGraph and agent design. First explain why linear chains aren't enough for agents and how a graph (state, nodes, edges, conditional routing) solves that – contrast a chain vs a graph clearly. Then guide me to build a minimal graph: define a state object, add two nodes, connect them, and add one conditional edge. Next, help me build a tool-using agent that decides whether to call a tool and loops until it can answer. Explain how state flows through the graph at each step.

Day 9 (Thursday) — MCP & MCP Servers

DAY 9 · THURSDAY — MCP	
10:30–11:30	Concept: What the Model Context Protocol (MCP) is — a standard way to give models tools, resources, and prompts. Hosts, clients, and servers. Why a standard matters.
11:30–1:30	Lab: Install the MCP SDK. Explore an existing MCP server. Connect an MCP server to a host (e.g. Claude Desktop / Claude Code) and inspect its tools.
2:30–4:30	Project: Build your own MCP server that exposes 2–3 tools (e.g. a calculator, a file reader, or an API wrapper).
4:30–6:00	Practice: Add a 'resource' to your server. Test each tool from a host. Handle errors gracefully.
6:00–6:30	Wrap-up: Commit code. Prepare — Friday is the Week 2 assessment (first 3 hours).

Learning Sources

- [Model Context Protocol — Introduction](#) — the official spec & concepts.
- [MCP — Build a Server \(Quickstart\)](#) — step-by-step server build.
- [MCP Python SDK \(GitHub\)](#) — the SDK and examples.
- [Anthropic — Introducing MCP](#) — the announcement & motivation.

Claude Code Prompt — Day 9

Teach me the Model Context Protocol (MCP). Explain what problem it solves, the roles of host, client, and server, and the three things a server can expose: tools, resources, and prompts. Use a plug-and-socket analogy. Then guide me to build my own MCP server in Python using the official SDK that exposes two or three tools (for example a calculator and a file reader). Explain how to test it from an MCP host and how to handle errors. Walk me through it step by step and let me write the tool functions myself.

Day 10 (Friday) — Capstone Integration + Week 2 Assessment

DAY 10 · FRIDAY — Assessment + Integration	
10:30–1:30	WEEK 2 ASSESSMENT: Practical build combining RAG + LangChain/LangGraph + MCP (see Section 4).
2:30–4:30	Concept + Project: Putting it together — how RAG, agents, MCP, and a React front-end form a complete AI system. Review the reference architecture for the upcoming agent-building phase.
4:30–6:00	Practice: Plan a personal mini-project for Week 3. Draft its architecture and the Claude Code prompts you'll use.
6:00–6:30	Wrap-up: Demo what you built. Set goals for the agent-building phase ahead.

Learning Sources

- [Claude Code — Documentation](#) — your primary tool for the next phase.
- [Anthropic — Building Effective Agents](#) — agent design patterns.
- [LangGraph — Multi-Agent Systems](#) — scaling to multiple agents.

Claude Code Prompt — Day 10

Help me see the big picture. Explain how everything I learned – Python, React, prompting, RAG, LangChain, LangGraph, and MCP – fits together into one complete AI system. Draw the reference architecture in text (front-end, agent orchestration, retrieval, tools via MCP). Then help me plan a personal mini AI-agent project for the next phase: define the goal, the architecture, the tools it needs, and a checklist of Claude Code prompts I'll use to build it.

4. Friday Assessment Program

Assessment Principle

Assessments are practical builds, not written exams. No question paper is handed out.

Each Friday, the first 3 hours (10:30 AM – 1:30 PM) are the assessment window.

Interns are given a brief, build a working solution, and demo it.

Evaluation is on what works, code quality, and the intern's ability to explain choices.

Week 1 Assessment — Friday, Day 5 (10:30 AM – 1:30 PM)

Theme: Foundations & Front-end. The intern builds a small full-stack-flavoured app within the 3-hour window.

The Brief

Build a working application that demonstrates Python and React skills together. A suggested brief (the trainer may swap the data source):

1. A Python script or small API that fetches and processes data from a public API, with validation and error handling.
2. A React front-end (Vite) that displays the data with at least: a list of items, a search/filter box, and loading & error states.
3. Clean component structure, sensible state management, and a short README explaining how to run it.

What is Evaluated

Criterion	Weight	Looking for
Functionality — does it work?	35%	App runs, core features work
Code quality & structure	25%	Readable, well-named, modular
React fundamentals	20%	Correct props/state/hooks use
Python fundamentals	10%	Idiomatic, handles errors
Explanation & demo	10%	Can justify decisions

Allowed during assessment

Official documentation and the learning sources in this guide.

Claude Code may be used as an assistant — interns must still understand and explain every line.

The trainer may ask the intern to modify a feature live to confirm understanding.

Week 2 Assessment — Friday, Day 10 (10:30 AM – 1:30 PM)

Theme: AI Systems & Orchestration. The intern builds a working AI system within the 3-hour window.

The Brief

Build an AI application that demonstrates the Week 2 stack. A suggested brief (the trainer may adjust scope):

1. A RAG pipeline: ingest a provided document set, chunk and embed it, and answer questions with source citations.
2. Use LangChain and/or LangGraph to structure the application — a retrieval chain, or a small agent that decides when to retrieve.
3. Expose at least one capability as a tool, ideally via a simple MCP server, or integrate one tool into a LangGraph agent.
4. A short README and a live demo showing the system answering questions correctly.

What is Evaluated

Criterion	Weight	Looking for
RAG correctness	30%	Relevant retrieval, grounded answers
Orchestration (LangChain/LangGraph)	25%	Clean, sensible composition
Tooling / MCP integration	20%	A working tool or MCP server
Code quality & structure	15%	Readable, modular, documented
Explanation & demo	10%	Can justify architecture

Assessment day flow (both Fridays)

10:30 AM

Trainer hands out the brief and any data files. Intern plans their approach (15 min).

10:45 AM – 1:00 PM

Build time. Trainer is available for clarifying questions only.

1:00 – 1:30 PM

Each intern demos their build and answers questions on their choices.

5. Learning with Claude Code

Claude Code is a command-line tool that lets interns delegate and pair-program with Claude directly from the terminal. During this bootcamp it serves two roles: a tutor for self-study, and a preview of the AI-assisted workflow interns will use in the agent-building phase.

Setup

Claude Code requires Node.js. After installing it, interns should verify it runs and log in. The official setup guide is the source of truth:

- [Claude Code — Setup & Quickstart](#) — installation, login, and first commands.

How to Learn with Claude Code

The daily prompts in this guide are starting points. To get the most from Claude Code as a tutor:

- Ask it to teach, not just to solve. Phrases like 'explain step by step' and 'let me write the code first' turn it into a tutor.
- Ask for analogies when a concept is abstract (embeddings, async, graphs).
- After it explains, ask it to quiz you — active recall beats passive reading.
- When you hit an error, paste the full error and ask 'explain why this happened' before asking for the fix.
- Ask it to review your code for 'common beginner mistakes' in that technology.
- Always type out and run the code yourself — understanding comes from doing.

Important — use Claude Code to learn, not to skip learning

Claude Code can write whole projects. During the bootcamp, the goal is your understanding. If Claude Code writes something, ask it to explain every part until you could rewrite it yourself. In Friday assessments you must explain and modify your code live — there is no shortcut.

Reusable Tutor Prompt Template

Interns can adapt this template for any topic not covered by the daily prompts:

Generic Self-Study Prompt

I am an intern learning [TOPIC]. I already know how to program but I'm new to this. Act as my mentor:

1. Explain [TOPIC] from first principles, with an analogy.
 2. Break it into 3-5 key sub-concepts and teach them one at a time.
 3. After each sub-concept, give me a small exercise and wait for my answer.
 4. Then guide me to build a small project using [TOPIC] – step by step, letting me write each piece before you show yours.
 5. Review my code for common mistakes and explain idiomatic patterns.
- Do not dump all the code at once. Teach me so I could rebuild it alone.

Debugging Prompt Template

Generic Debugging Prompt

I hit an error while building [WHAT]. Here is the full error message and the relevant code:

[PASTE ERROR]

[PASTE CODE]

First explain WHY this error happened in plain language. Then explain how to fix it and how I could have caught it earlier. Don't just give me the fixed code – make sure I understand the cause.

6. After the Bootcamp — The Agent-Building Phase

Once the two weeks are complete, interns transition from learning to building. They will work on real AI agents and systems, using AI tools like Claude Code as their daily engineering partner.

What Changes

Aspect	Bootcamp (Weeks 1–2)	Building Phase (Week 3+)
Goal	Learn the stack	Ship working agents & systems
Claude Code role	Tutor — explains everything	Pair programmer — accelerates delivery
Work	Guided projects & labs	Real project requirements
Assessment	Friday practical builds	Project milestones & demos

Readiness Checklist

Before starting the building phase, each intern should be able to honestly tick every box:

- I can set up a Python project, manage dependencies, and call external APIs safely.
- I can build a React front-end with components, state, hooks, and API integration.
- I can write clear, structured prompts and know when to use each pattern.
- I can build a RAG pipeline and explain every parameter that affects it.
- I can compose an LLM app with LangChain and orchestrate an agent with LangGraph.
- I understand MCP and have built a working MCP server.
- I can use Claude Code to plan, build, and debug — and explain everything it produces.

Final note for interns

These two weeks are intense by design. The pace mirrors how fast this field moves. Ask questions early, commit code daily, and treat every blocker as a learning opportunity. By Day 11 you won't just know the tools — you'll be ready to build with them.